Computer Programming: CT

Why do we need programming languages?

Programming languages are needed to communicate between humans and computers.

Humans think in natural language, which can be ambiguous. But computers understand only machine language (0 and 1), which is exact and clear.

Since machine language is difficult for humans to understand, programming languages act as a bridge between human and machine.

They use strict rules (syntax and semantics) and can be translated into machine language by a compiler or interpreter.

There is a big gap between human thinking and machine understanding.

Natural Language: Ambiguous ("I saw the man with the telescope").
Machine Language: Exact (01001000).
Programming Language: A formal compromise (Strict rules, readable by humans).

What is Source Code?

Source code is the program written by a programmer in a high-level programming language such as C++, Java, or Python.

It is human-readable and follows specific grammar rules called syntax.

Source code cannot be executed directly by the hardware. It must be translated into machine language by a compiler or interpreter.

Example (Python):

```
print("Hello, World!")
```

What is Machine Code?

Machine code is the lowest level of software instructions.

It consists entirely of binary digits (0 and 1).

Machine code directly controls the CPU (Central Processing Unit).

It is hardware-specific, meaning machine code for Intel processors is different from ARM processors.

Example (Binary for "He"):

```
01001000 01100101
```

Only janar jonno

ARM এর পূর্ণরূপ হলো **Advanced RISC Machine**।
এটি একটি প্রসেসরের (CPU architecture) ধরন।
ARM প্রসেসর সাধারণত মোবাইল ফোন, ট্যাবলেট এবং এমবেডেড সিস্টেমে ব্যবহৃত হয়, কারণ এটি কম বিদ্যুৎ খরচ করে এবং কার্যকরভাবে কাজ

Alhamdulillah খুব ভালো লাগলো শুনে।
নিচে তোমার পুরো বিষয়টা **exam-friendly English version** করে দিলাম — simple, clear, and easy to write in exam.

Translator

A **Translator** is a system software that converts High-Level Source Code into Low-Level Machine Code.

```
Source Code (C++/Python)  →  Translator  →  Machine Code(010101)
```

There are two main types of translators:
1. Compiler 2.Interpreter

1. Compiler

A **Compiler** translates the entire program at once and creates a separate executable file.

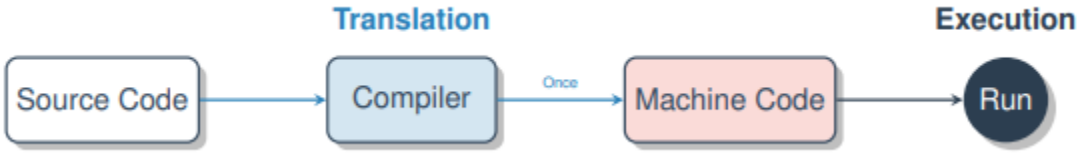
Process:
Read all code → Check errors → Build executable file

Analogy:
Like a translator who translates a whole English book into Bangla before anyone reads it.

Pros:
✓ Execution is very fast (CPU runs the binary directly).

Cons:
✗ If one line is changed, the whole program must be recompiled.

Compiler Execution Flow



2. Interpreter

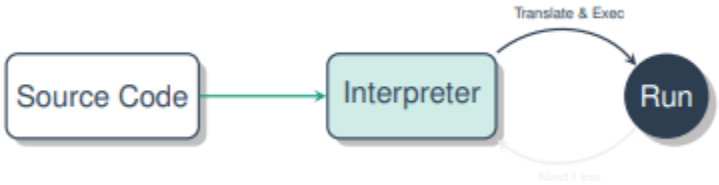
An **Interpreter** translates and executes the program line-by-line.

Process:
Read Line 1 → Execute
Read Line 2 → Execute

Analogy:

Like a live interpreter who translates a speech while it is being delivered.

Pros:
✓ Easy to debug (stops exactly where the error occurs).



Types of Errors

During program execution, different types of errors can occur.

There are mainly three types of errors:

Syntax Error

A **Syntax Error** happens when the grammar rules of the programming language are violated.

It is detected by the **compiler or interpreter**.

Example:
Missing a semicolon (;)

Logical Error

A **Logical Error** occurs when the program runs successfully but produces the wrong result.

It is usually found by the **programmer (human)**.

Example:
Calculating `2 + 2 = 3` instead of 4

Runtime Error

A **Runtime Error** happens while the program is running and may cause the program to crash.

Example:
Dividing a number by zero.

Algorithm:

An algorithm is a step-by-step procedure or a set of rules to solve a problem. It is **language independent**, which means you can write it in simple English first and then implement it in any programming language like **C++, Java, or Python**.

Example (Making Tea Algorithm):

1. Boil water.
2. Add tea leaves.
3. Pour into a cup.
4. Add sugar.

Note: An algorithm tells **what to do step by step**, not the actual code.

Characteristics of a Good Algorithm:

1. **Input:** Can take zero or more inputs.
2. **Output:** Must give at least one result.
3. **Definiteness:** Every step should be clear and easy to understand.
4. **Finiteness:** Must finish after a limited number of steps.
5. **Effectiveness:** Steps should be simple and doable.

Pseudocode

Pseudocode (False Code) is a way of describing an algorithm using a mix of plain English and code-like structure.
It ignores strict syntax (no semicolons needed).
It helps programmers plan logic before worrying about grammar.

Example: Add Two Numbers

```
START
READ number1, number2
sum = number1 + number2
PRINT sum
END
```

Control Flow Graph (CFG):

A CFG shows **all possible paths** a program can take during execution.

Parts:

1. **Nodes:** Represent **basic blocks** (a sequence of instructions without jumps).
2. **Edges:** Show **control flow** (jumps or branches).

Rules:

1. Assign a **node number** to each expression.
2. Draw a **directed edge A → B** if B executes after A.
3. For branches, mark edges as **True or False**.

Problem:

Write a program to check if a student Passed or Failed. (Pass Mark = 40).
We will look at: 1 The Algorithm. 2 The Pseudocode. 3 The Control Flow Graph.

Algorithm (English)

1. Start.
2. Ask student for marks.
3. Check if marks are 40 or more.
4. If yes, say "Pass".
5. If no, say "Fail".
6. Stop.

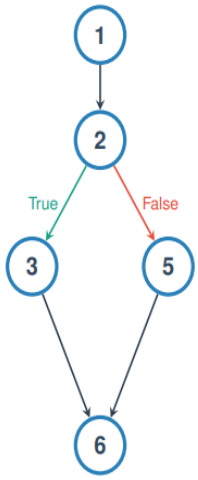
Pseudocode

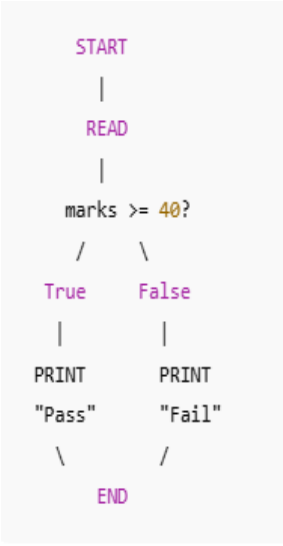
```
START
INPUT marks
IF marks >= 40 THEN
  PRINT "Pass"
ELSE
  PRINT "Fail"
ENDIF
END
```

Process of Creating a CFG

Sample Code:

```
(1) input x
(2) if x > 10:
(3)   print "High"
(4) else:
(5)   print "Low"
(6) print "End"
```





Big-O Notation

Definition:

- **Big-O notation (O)** shows the **worst-case scenario** of an algorithm.
- It tells the **maximum time** an algorithm can take.

Notation:

- If f(n)=O(g(n)), then g(n) represents the **upper bound** of time complexity.

Example – Linear Search:

- **Problem:** Find value **7** in the list [1,2,3,4,5,6,7]
- **Worst Case:** The number is at the **end** of the list.
- **Work:** Must check **all n elements**.
- **Result:** O(n)⟶ Linear time.

Key Point:

Big-O always considers the **maximum work** an algorithm can do, not the best case.

Big-Omega Notation

Definition:

Big-Omega (Ω) shows the **best-case scenario** of an algorithm. It tells the **minimum time** an algorithm requires.

Example – Linear Search:

- **Problem:** Find value **1** in the list [1,2,3,4,5]
- **Best Case:** The number is at the **first index**.
- **Work:** Check **only 1 element**.
- **Result:** Ω(1)→ Constant Time.

Big-Theta Notation (Θ)

Definition:

- **Big-Theta (Θ)** shows the **average or tight bound** of an algorithm.
- It gives both the **upper bound and lower bound** of time complexity.

When to Use Θ?

1. When **Best Case and Worst Case are different** (like Linear Search), Θ is more complex to determine.
2. When an algorithm always takes the **same time** for every input, then: O(1)=Ω(1)=Θ(1)

Example:

- Accessing an array index always takes constant time.
- So, its complexity is **Θ(1)**.

How to Calculate Complexity (Code Example)

Question: Calculate the Time Complexity of the following snippet.

```
void f(int n) {
    int a = 0;            // Cost: 1
    for (int i=0; i<n; i++) {
        a = a + 1;        // Cost: n
    }
    for (int j=0; j<n; j++) {
        for (int k=0; k<n; k++) {
            cout << k;    // Cost: n * n
        }
    }
}
```

Analysis:

$$Total = 1 + n + n^2$$

Rules for Big-O:

1. Drop constants ($1 \to 0$).
2. Drop lower order terms (n is smaller than n^2).
3. Keep the dominant term.

Answer: $O(n^2)$ **(Quadratic)**

The Compilation Process

When we click "Build" in C++, the source code goes through three main stages:

1. Preprocessing

- Handles all commands starting with #
- Example: `#include <iostream>`

- It copies header file contents into the program.
- Cleans the code for the compiler.

2. Compiling

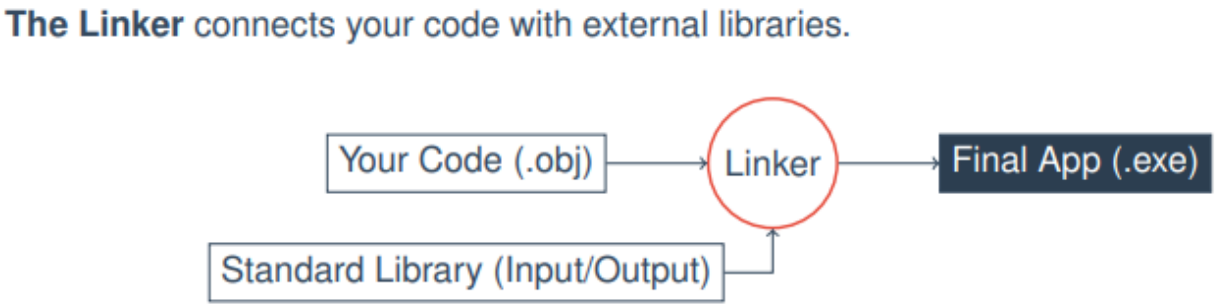
- Converts source code into **machine code (Object file)**
- Checks for **syntax errors**
- If no error → generates an **Object file (.obj or .o)**
- This file is incomplete (missing libraries).

3. Linking

- The **Linker** connects your object file (.obj / .o) with **external libraries**.
- It combines all required files to create the **final executable file (.exe)**.

Example:

- When you use `cout` (print), the printing code is not inside your file.
- The **Linker takes that code from system libraries** and attaches it to your program.



Question:

1. Statement dibe solve korte hobe tarmane Algorithm design korte hobe.

2. Algorithm dibe pseudocode likhte hobe

3. Algorithm to Pseudocode to CFG korte dibe.